Lecture 6.

Python Protocols, Iterators & Functional Abstractions

Software Development & Python
Ivan Solomatin
MIPT

Outline

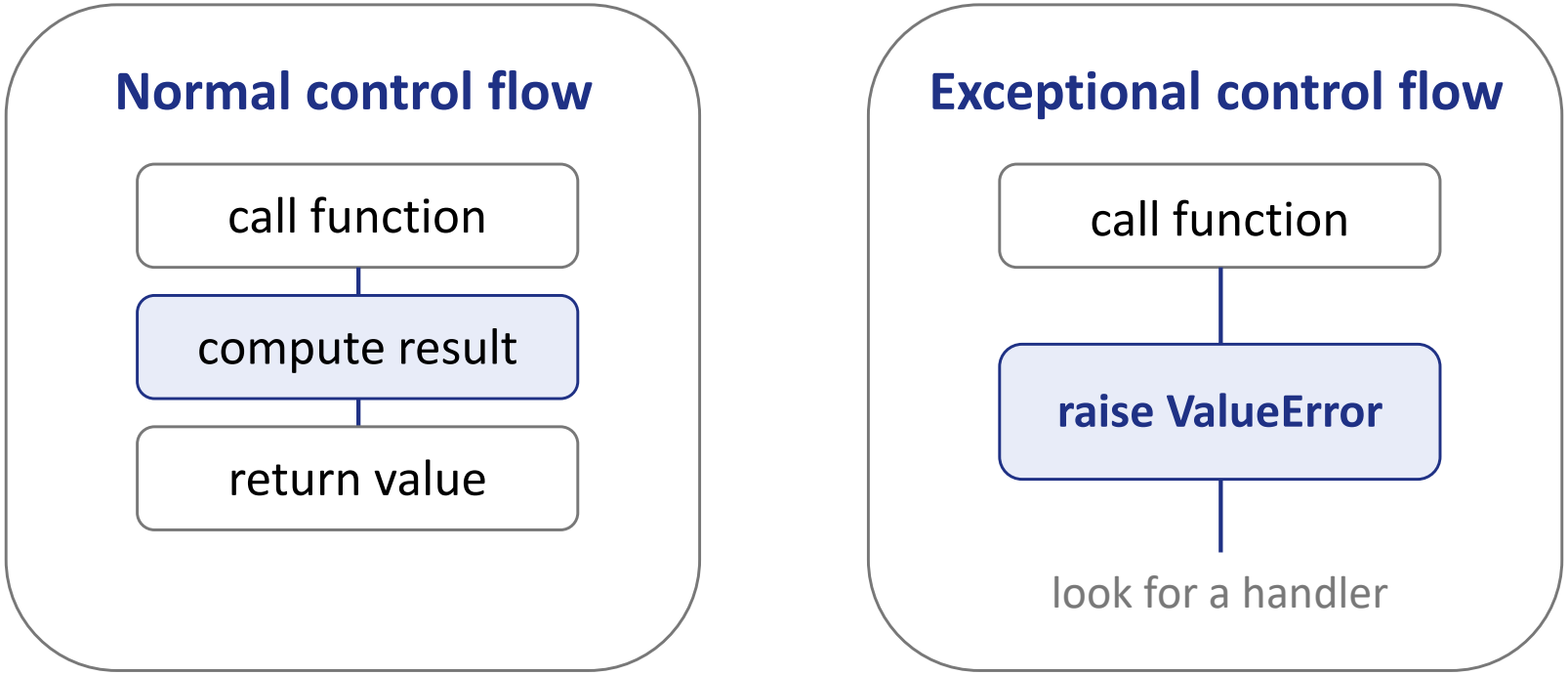
- Exceptions
- Protocols and Python syntax
- Iterable vs iterator
- Generators and lazy evaluation
- Functions as first-class objects
- Closures and decorators
- Comprehensions and generator expressions
- Context managers
- Putting Python protocols together

Exceptions

Errors become objects — and change control flow

Exceptions

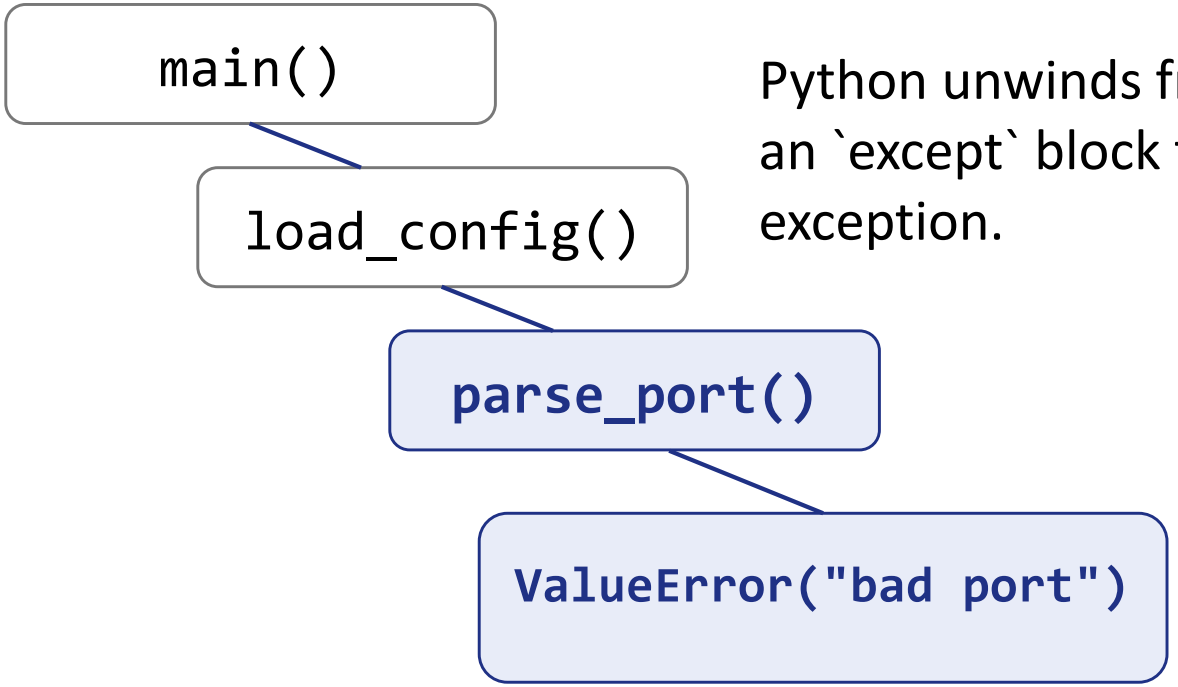
What is an exception?



An exception is an object that carries information about a failure.

Exceptions

Exceptions propagate through the call stack



Python unwinds frames until it finds an `except` block that matches the exception.

If nobody handles it, the program terminates and Python prints a traceback.

Exceptions

Handling an exception: try / except

```
try:
    port = int(text)
    connect(port)
except ValueError as err:
    print("Invalid port:", err)
```

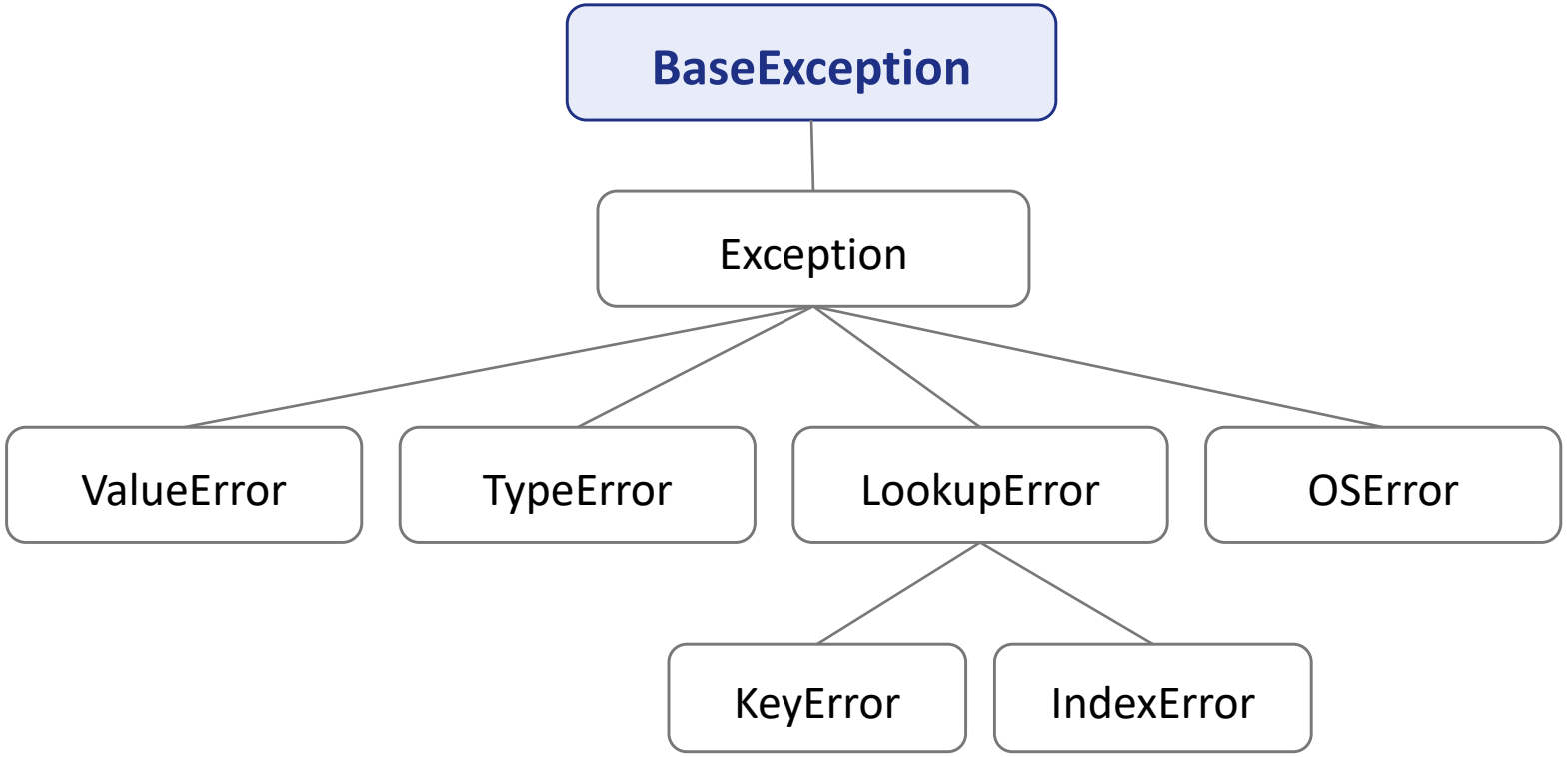
Catch what you can handle

- `try` contains code that may fail.
- `except ValueError` catches only that exception type (and its subclasses).
- After the handler finishes, execution continues after the whole try statement.

Prefer specific exception types to a broad `except:`.

Exceptions

Exceptions form a class hierarchy



`except Exception:` catches most ordinary application errors — but not every BaseException.

SystemExit and KeyboardInterrupt intentionally live outside the usual Exception branch.

Exceptions

else and finally

`try`

code that may raise

`else`

runs only if try succeeded

`finally`

runs when leaving the block (always)

```
try:
    resource = open_file()
except OSError:
    report_failure()
else:
    use(resource)
finally:
    cleanup()
```

`finally` is the foundation of reliable cleanup — even when an exception occurs.

Exceptions

Your code can raise exceptions too

```
def set_volume(value):
    if not 0 <= value <= 100:
        raise ValueError(
              "volume must be "
              "in [0, 100]")
```

Raise when a contract is violated

- `raise` creates exceptional control flow intentionally.
- Choose an exception type that describes the problem.
- The message should help the caller understand what was wrong.

Exceptions are part of an API: callers need to know what failures to expect.

Exceptions

Custom exceptions

```
class InsufficientMana(Exception):
    pass


if spell.cost > player.mana:
    raise InsufficientMana()
```

Why define your own?

- It gives a domain-specific failure a precise name.
- Callers can catch it without confusing it with unrelated errors.
- Usually inherit from `Exception`.
- You may store additional information needed for logging/fixing the source of the error.

What if an exception happens while a resource is open?

Context managers (`with`) package cleanup logic so it works on both normal and exceptional paths.

Protocols

Protocols

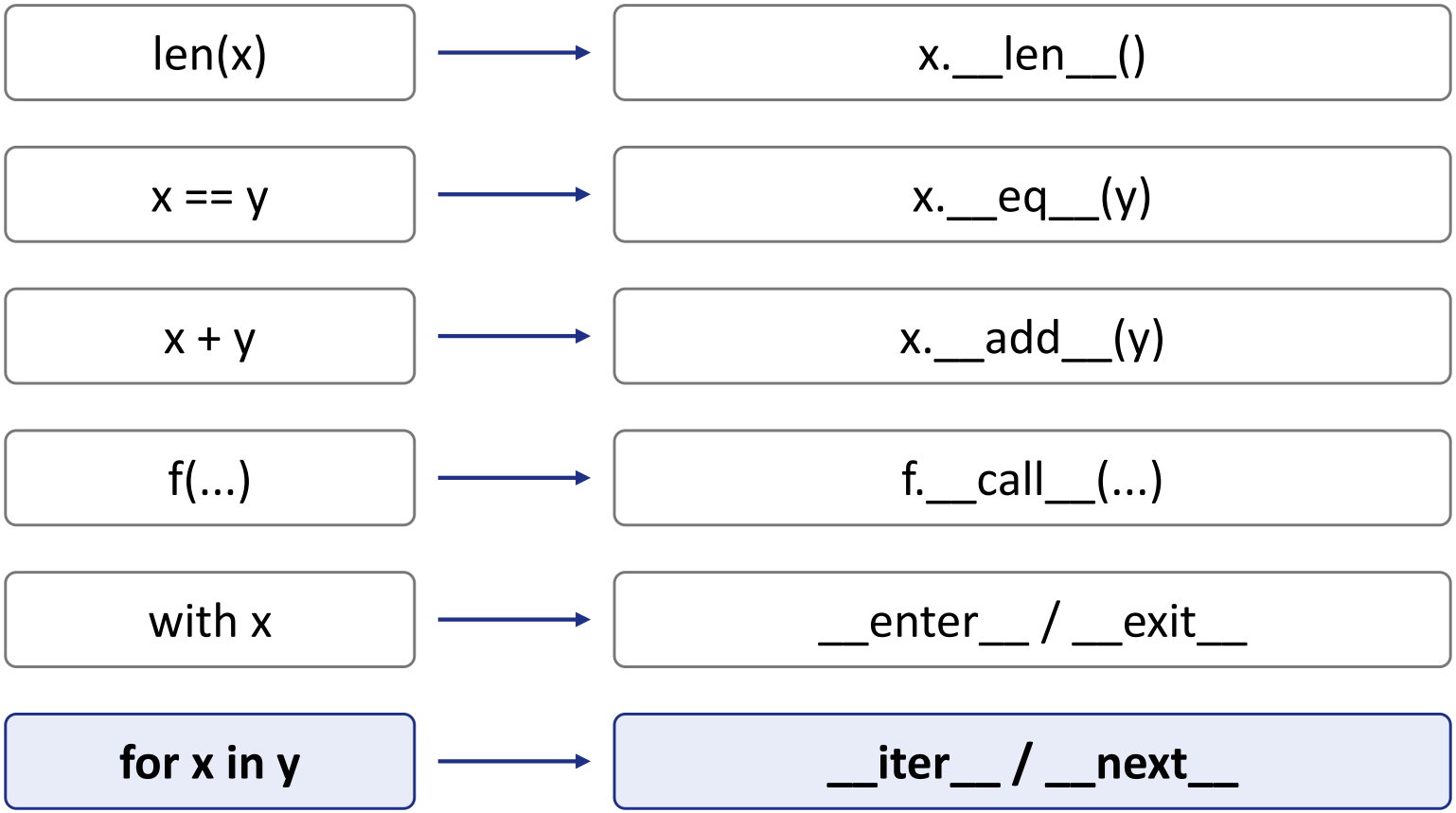
```
len(obj)
obj == other
for x in obj:
    ...
```



These operations are implemented through object protocols.

Python syntax often delegates behavior to objects.

Protocols



A protocol is a contract expressed through operations, not necessarily inheritance.

Protocols

Duck typing with structure

```
def total_length(items):
    return sum(len(x)
               for x in items)

items = [
  [1, 2, 3],
  ('a', 'b'),
  'cde',
  {}
]
print(total_length(items)) # 8
```

str

list

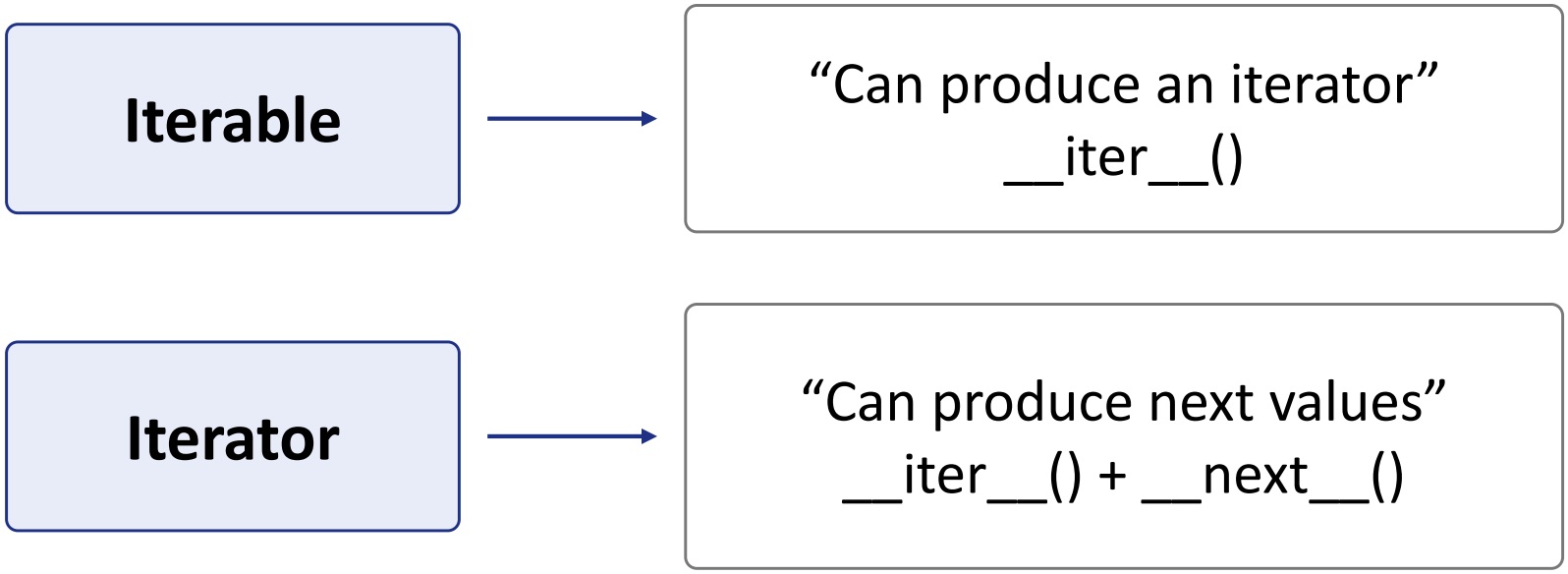
tuple

custom object with __len__

The function asks for an operation, not a particular class.

Protocols

A protocol defines expected operations



Protocols let unrelated types participate in the same language construct.

Iteration protocol

What really happens in a for loop?

Iteration protocol

The same for loop works on different objects

```
x = [1, 2, 3]
for v in x:
    print(v)
```

```
s = 'abc'
for ch in s:
    print(ch)
```

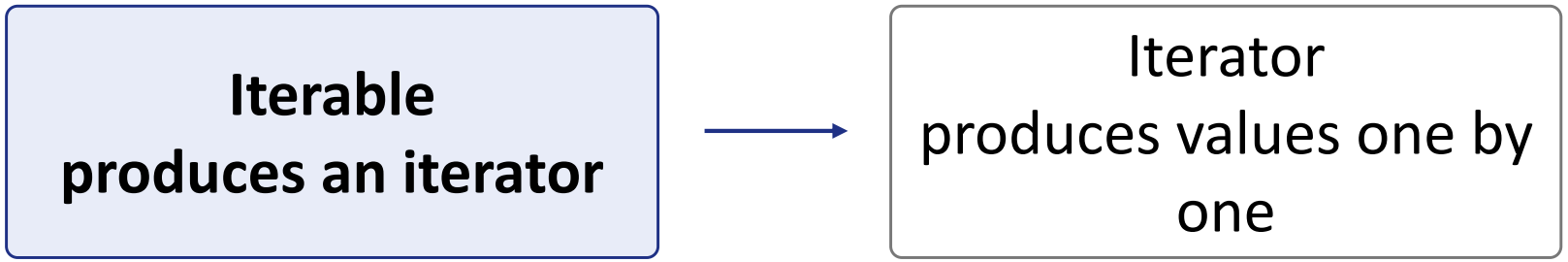
```
f = open('input.txt')
for line in f:
    process(line)
```

Different types — same iteration protocol

for does not need to know the concrete container type.

Iteration protocol

Iteration has two roles



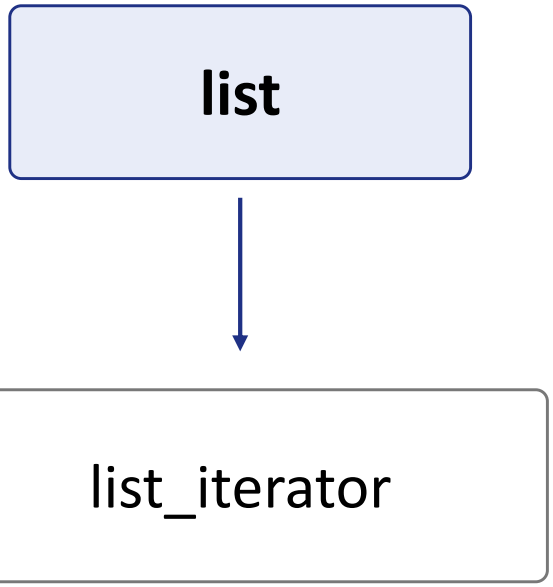

```
it = iter(values) # values -- iterable
next(it)          # it -- iterator
next(it)
```

iterable ≠ iterator

Iteration protocol

Step 1: `for` gets an iterator with `iter()`

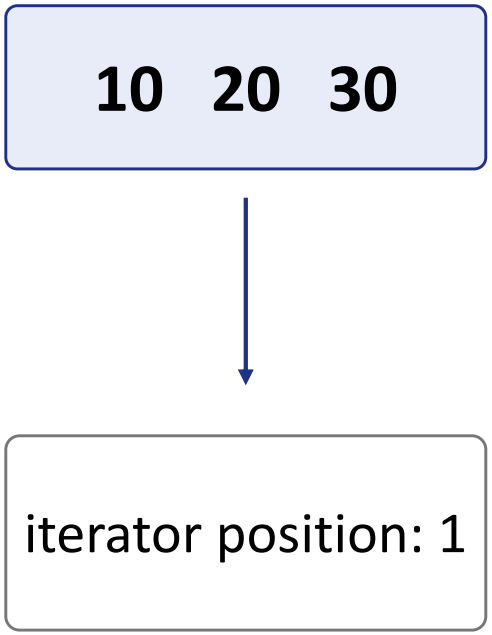
```
values = [10, 20, 30]
it = iter(values)
```



The iterable can create an iterator – separate object that stores state of iteration.

Iteration protocol

Step 2: next() changes iterator state




```
next(it)  # 10
next(it)  # 20
```

iterator position: 2

An iterator is stateful.

Iteration protocol

How does iteration end?

```
it = iter([10, 20])
next(it)  # 10
next(it)  # 20
next(it)  # StopIteration
```

StopIteration signals "no more values"

for catches StopIteration for us.

Iteration protocol

A for loop is protocol-driven

```
for x in
values:
    print(x)
```



```
it = iter(values)
while True:
    try:
        x = next(it)
    except
StopIteration:
        break
    print(x)
```

The language construct is built on a small protocol.

Iteration protocol

We can implement the protocol ourselves

```
class Countdown:
    def __init__(self, start):
        self.current = start

    def __iter__(self):
        return self

    def __next__(self):
        if self.current == 0:
            raise StopIteration
        value = self.current
        self.current -= 1
        return value
```

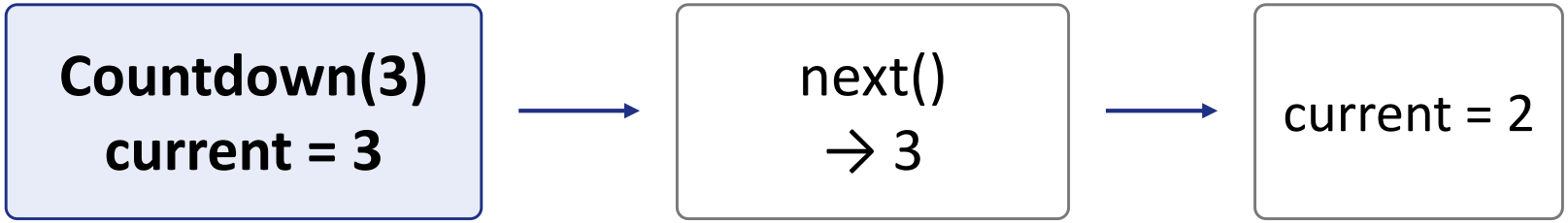
```
for x in Countdown(3):
    print(x)

# 3
# 2
# 1
```

The class is both iterable and iterator.

Iteration protocol

The iterator object stores execution state



The state lives between calls to next().

Generators

A simpler way to write stateful iteration

Generators

yield

`yield` turns a function into a generator function

```
def countdown(start):
    while start > 0:
        yield start
        start -= 1
```

```
g = countdown(3)
next(g)  # 3
next(g)  # 2

g = countdown(3)
for v in g:
  print(v)
```

The generator object remembers where execution paused.

Generators

Creating generator

```
def countdown(start):
    while start > 0:
        yield start
        start -= 1
```

g = countdown(3)

Calling the function creates a generator object.
The body has not run yet.

Generator call ≠ immediate execution.

Generators

First next()

```
next(g)
```



```
while start > 0:
    yield start   # → 3
    start -= 1
```

Execution pauses at yield.
Local state is preserved.

Generators

Resuming

```
next(g)
```



```
start -= 1    # 2
```

Execution resumes after the previous yield.

Generators

Iterator class vs generator

Iterator class

- explicit state attributes
- __iter__
- __next__
- StopIteration
- more control

Generator function

- local variables are state
- yield
- automatic protocol
- concise
- usually easier

Generators are syntactic support for writing iterators.

Lazy evaluation

Compute values only when they are needed

Lazy evaluation

List comprehension vs generator expression

```
squares = [x * x for x in range(1000000)]
```

Build all values now (and store them in memory)

```
squares = (x * x for x in range(1000000))
```

Produce values on demand

Lazy evaluation

Lazy computation changes the memory model

Eager list

1 2 3 4 5 6 7

Many values stored at once

Lazy generator

state + next value

Values can be discarded after use

```
squares = [
x * x for x in range(10**18)
]

for s in squares:
    if s > 100:
        break
    print(s)
```

Fails (MemoryError)

```
squares = (
x * x for x in range(10**18)
)

for s in squares:
    if s > 100:
        break
    print(s)
```

Works normally

Lazy ≠ always faster; lazy means deferred and incremental.

Lazy evaluation

Generators may be infinite

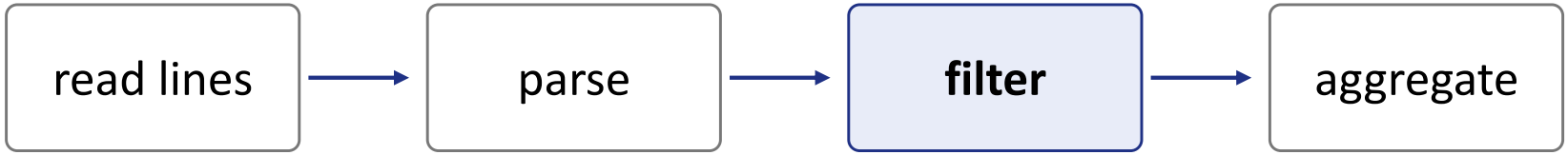
```
def naturals():
    n = 0
    while True:
        yield n
        n += 1
```

```
for n in naturals():
    if n > 5:
        break
    print(n)
```

An infinite iterator is fine if consumers stop.

Lazy evaluation

Lazy iterators compose into pipelines




```
rows = (parse(line) for line in file)
valid = (r for r in rows if r.ok)
total = sum(r.value for r in valid)
```

Each stage can process one value at a time.

Functions are objects

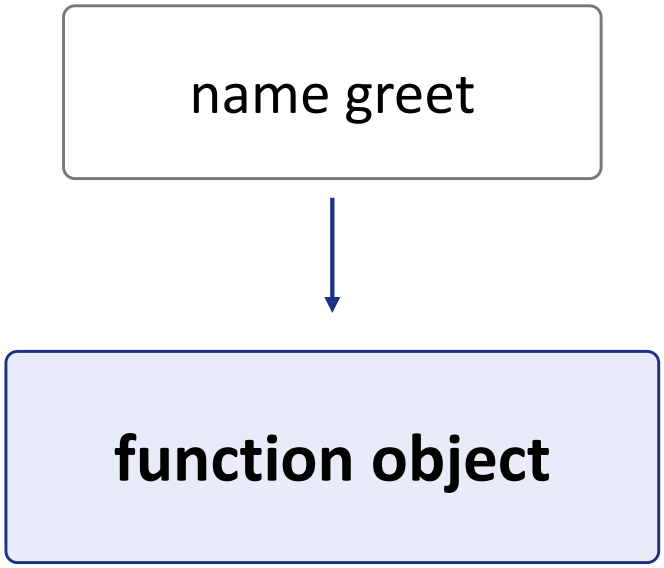
Values can represent behavior

Functions are objects

A function can be stored like any other object

```
def greet(name):
    return f"Hello, {name}"

f = greet
print(f("Alice"))
```



Names can refer to functions too.

Functions are objects

Behavior can be passed as an argument

```
def apply_twice(f, x):
    return f(f(x))

def inc(x):
    return x + 1

apply_twice(inc, 10) # 12
```

data: 10

behavior: inc

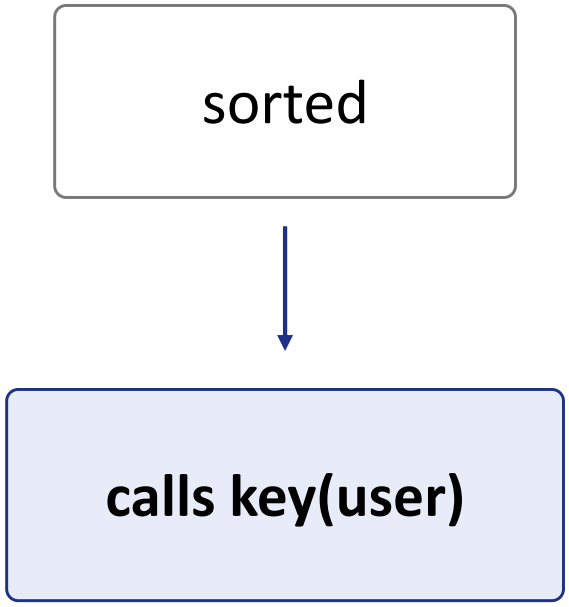
Functions let us parameterize algorithms by behavior.

Functions are objects

A practical higher-order function: sorted(key=...)

```
users = [
    {"name":"Bob",
     "age":31},
    {"name":"Alice",
     "age":27},
]

sorted(users, key=lambda
u: u["age"])
```



key is a function object describing how to compare items.

Closures

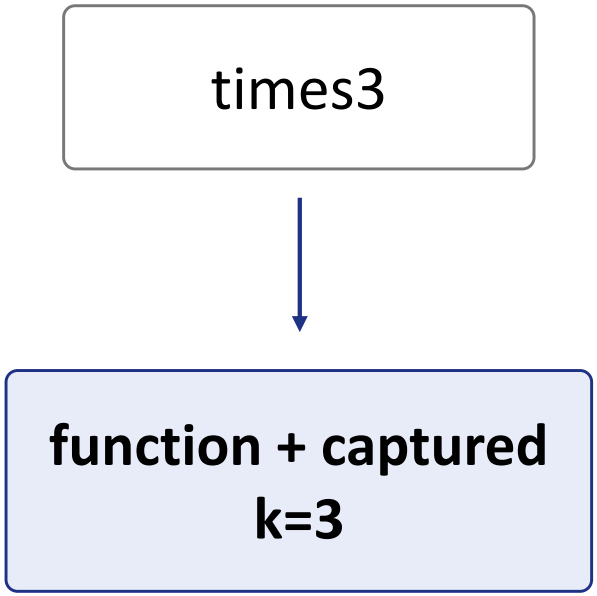
Functions can remember values from an enclosing scope

Closures

A nested function can capture surrounding state

```
def make_multiplier(k):
    def multiply(x):
        return x * k
    return multiply

times3 = make_multiplier(3)
times3(10)  # 30
```



The enclosing call has finished, but the captured value remains available.

Closures

`nonlocal` lets a closure update captured state

```
def make_ctr():
    count = 0

    def next_value():
        nonlocal count
        count += 1
        return count

    return next_value
```

```
counter = make_ctr()
counter()  # 1
counter()  # 2
counter()  # 3
```

Use nonlocal when the closure must update a binding, not just read it.

Closures

Where are closures useful?

Function factories	create specialized behavior
Callbacks	remember configuration
Decorators	wrap a function and keep references
Small stateful utilities	without defining a class

Closures and objects are two different ways to package state + behavior.

Decorators

Transform functions with functions

Decorators

A decorator takes a function and returns a function

```
def trace(f):
    def wrapper(*args, **kwargs):
        print("Calling", f.__name__)
        res = f(*args, **kwargs)
        print(f.__name__, "returned", res)
        return res
    return wrapper
```

```
def add(a, b):
    return a + b

add = trace(add)
add(2, 3)
```



```
Calling add
add returned 5
```

The original name is rebound to the wrapped function.

Decorators

@decorator is syntax for rebinding

```
def add(a, b):
    return a + b
add = trace(add)
```



```
@trace
def add(a, b):
    return a + b
```

The two forms mean the same transformation.

Decorators

Decorators separate cross-cutting behavior

```
from time import perf_counter

def timer(f):
    def wrapper(*args, **kwargs):
        t0 = perf_counter()
        result = f(*args, **kwargs)
        print(perf_counter() - t0)
        return result
    return wrapper
```

logging

timing

authorization

caching

Decorators

A parameterized decorator adds one more function layer

```
def repeat(n):
    def decorator(f):
        def wrapper(*args, **kwargs):
            for _ in range(n):
                f(*args, **kwargs)
        return wrapper
    return decorator
```

```
@repeat(3)
def hello():
    print("Hi")
```

repeat(3)
returns a decorator
which transforms the function

Context managers

Reliable setup and cleanup

Context managers

Many resources have the same lifecycle




```
f = open("data.txt")
try:
    process(f)
finally:
    f.close()
```

Context managers

with makes cleanup part of the protocol

```
f = open("data.txt")
try:
    process(f)
finally:
    f.close()
```



```
with open("data.txt") as f:
    process(f)
```



__enter__()

body executes

__exit__()

Cleanup happens even when the body raises an exception.

Context managers

Context-manager protocol for a custom object

```
class Timer:
    def __enter__(self):
        self.start = perf_counter()
        return self

    def __exit__(self, exc_type, exc, exc_tb):
        print(perf_counter() - self.start)
```

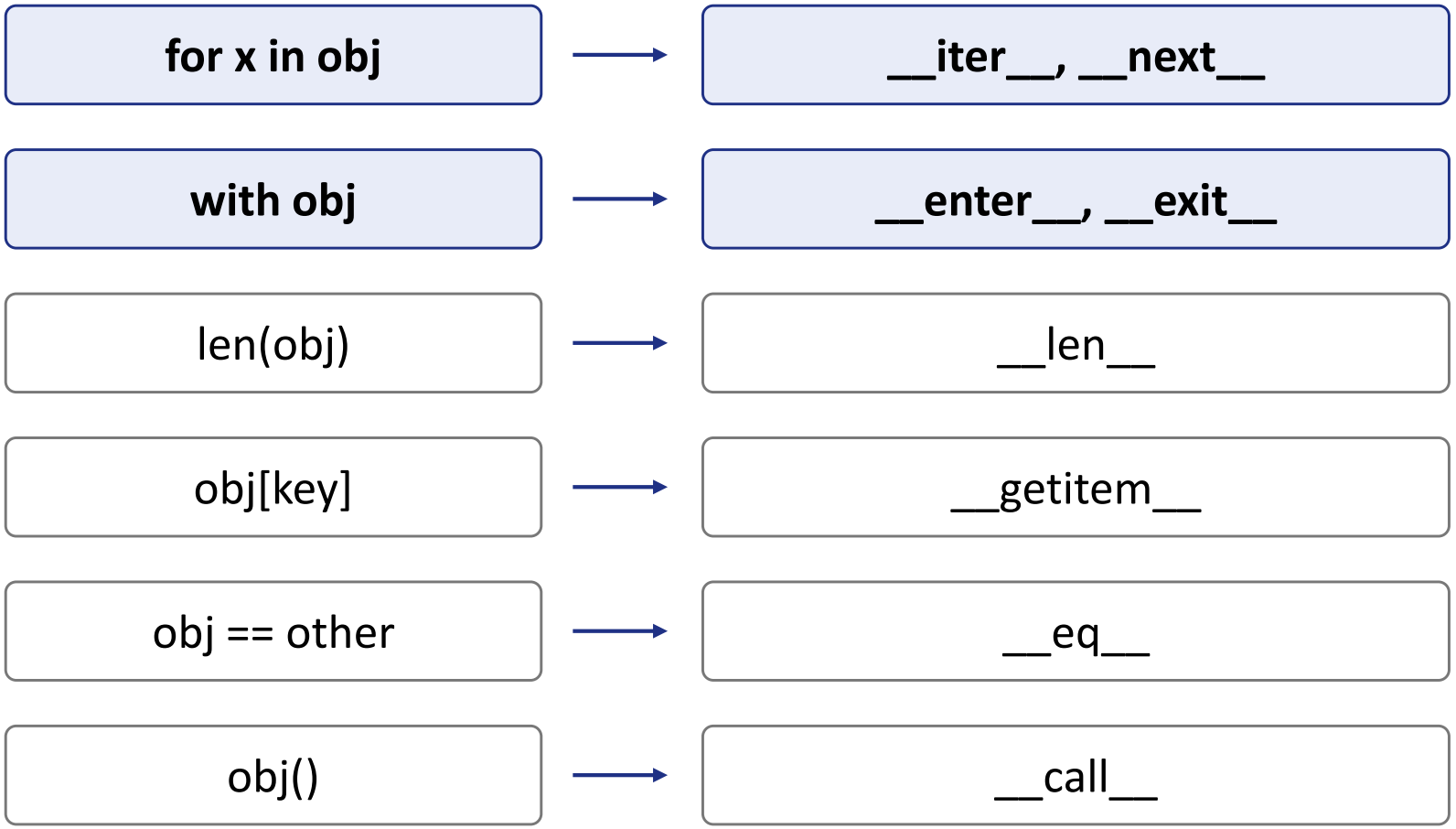
- exc_type — Type of exception;
- exc — value of exception;
- exc_tb — traceback.

```
with Timer():
    do_work()
```

Conclusion

Conclusion

Common syntax maps to protocols



Python syntax is often a friendly surface over protocol calls.

Conclusion

Even “function-like” behavior is a protocol

```
class Multiplier:
    def __init__(self, k):
        self.k = k

    def __call__(self, x):
        return x * self.k
```

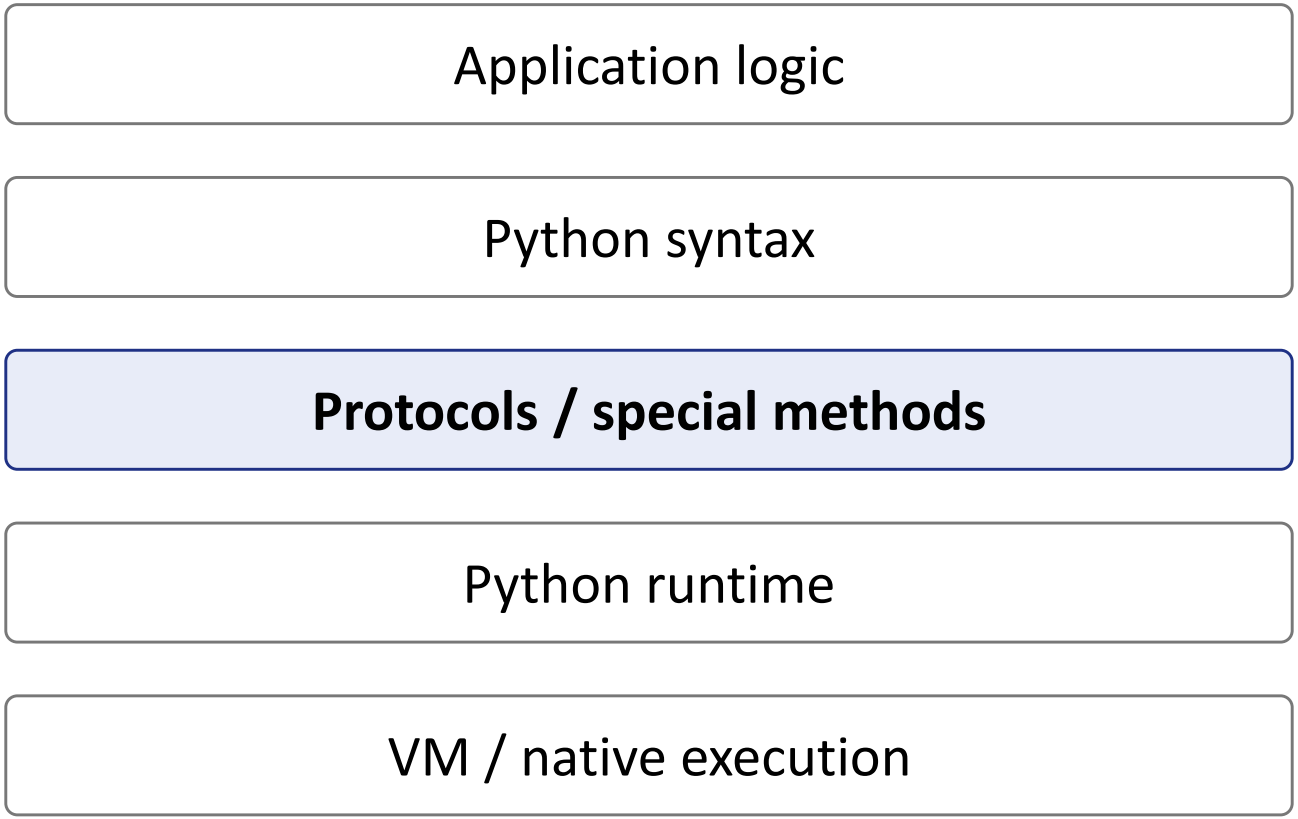
```
times3 = Multiplier(3)
times3(10)  # 30
```

object becomes callable

Classes and functions meet through shared protocols.

Conclusion

The abstraction stack keeps growing



High-level syntax works because lower layers provide reusable contracts.

Thank you for watching!